

Санкт-Петербургский государственный университет

Осовский Никита

Выпускная квалификационная работа

Частичная линейаризация слоёв внимания без дообучения

Уровень образования: бакалавриат

Направление 02.03.02 *«Фундаментальная информатика и информационные
технологии»*

Основная образовательная программа *СВ.5190.2022 «Большие данные и
распределенная цифровая платформа»*

Научный руководитель:
доцент кафедры фундаментальной информатики
и распределенных систем,
Ph. D.
Першин Антон

Рецензент:
доцент кафедры компьютерного моделирования
и многопроцессорных систем,
кандидат физико-математических наук
Корхов Владимир

Санкт-Петербург

2026

Saint Petersburg State University

Osovskiy Nikita

Bachelor's thesis

Partial Linearization of Attention Layers Without Fine-Tuning

Education level: Bachelor's degree

Field of study 02.03.02 “*Fundamental Computer Science and Information Technologies*”

Educational program SV.5190.2022 “*Big Data and Distributed Digital Platform*”

Scientific supervisor:
Associate Professor, Department of Fundamental
Computer Science and Distributed Systems,
Ph. D.
Pershin Anton

Reviewer:
Associate Professor, Department of Computer Modeling
and Multiprocessor Systems,
Candidate of Physical and Mathematical Sciences
Korkhov Vladimir

Saint Petersburg

2026

Содержание

Перечень терминов и определений	3
1 Введение	8
1.1 Предметная область и актуальность	8
1.2 Цель, гипотеза и задачи работы	9
2 Обзор литературы	11
2.1 Полное внимание и проблема длинного контекста	11
2.2 Линейное внимание и его ограничения	12
2.3 Post-hoc линейаризация: LoLCATs, LAWCAT и Lizard	13
2.4 Головы извлечения и сжатие KV-кэша	15
2.5 Замена блоков без дообучения: NBL	16
2.6 Выводы для настоящей работы	17
3 Предлагаемый метод	19
3.1 Концепция частичной гибридизации	19
3.2 Исследуемые конфигурации	22
4 Экспериментальная установка	23
4.1 Базовая модель и окружение	23
4.2 Программная реализация	24
4.3 Идентификация голов извлечения и построение масок	26
4.4 Протоколы оценки	27
5 Результаты и обсуждение	31
5.1 Базовый профиль пропускной способности	31
5.2 Валидация масок на NIAH	33
5.3 Сравнение четырёх архитектур	34
5.4 Диагностика обученных параметров LoLCATs	39
5.5 Сравнение с существующими результатами	40
6 Заключение	42
6.1 Подтверждение основной гипотезы	42
6.2 Практическая рекомендация	43

Список литературы	45
--------------------------	-----------

Перечень терминов и определений

В настоящей работе используются следующие термины и определения.

Адаптация без дообучения — модификация архитектуры или режима работы модели, при которой все исходные веса остаются замороженными. Допускается введение новых обучаемых параметров, но исходная модель не изменяется. Данная работа исследует адаптацию без дообучения.

Авторегрессионная генерация (*Autoregressive generation*) — режим работы LLM, при котором модель генерирует текст по одному токenu: на каждом шаге выбирается следующий токен на основе всех ранее сгенерированных, после чего он добавляется к контексту и процесс повторяется.

Большая языковая модель (*Large Language Model, LLM*) — нейросетевая модель на основе архитектуры трансформера, обученная на больших текстовых корпусах для задачи предсказания следующего токена.

Видеопамять (*Video RAM, VRAM*) — оперативная память графического ускорителя, используемая для хранения весов модели, KV-кэша и промежуточных активаций.

Вычислительное ядро (*GPU kernel*) — оптимизированная функция, выполняемая на графическом ускорителе. В контексте данной работы ядра — это высокопроизводительные реализации операций self-attention, линейного внимания и других вычислений, написанные для эффективного использования аппаратных ресурсов GPU.

Гибридизация внимания — подход, при котором в одной модели часть голов (или слоёв) сохраняет полное Softmax-внимание, а остальные переводятся в субквадратичный режим. Цель — получить компромисс между вычислительной эффективностью субквадратичных подходов и качеством извлечения информации, обеспечиваемым Softmax-вниманием.

Головы извлечения (*Retrieval heads*) — подмножество голов внимания, способных обращаться к далёким, но релевантным позициям контекста для извлечения конкретных фактов [8, 15].

Групповое внимание с общими ключами и значениями (*Grouped Query Attention, GQA*) — модификация МНА, в которой несколько голов запросов (Q) совместно используют одну пару ключ–значение (K, V). Такая группа голов называется *KV-группой*. Если в стандартном МНА каждая голова

имеет собственные K и V , то в GQA k голов запросов делят одну пару K , V . Это снижает объём хранимого KV-кэша пропорционально коэффициенту группировки k .

Дообучение (*Fine-tuning*) — процедура обновления весов предобученной модели на новых данных с целью адаптации к конкретной задаче.

Индукционная оценка (*Induction score*) — числовая мера способности головы внимания выполнять индукционный паттерн: обнаружив повторение ранее встреченной последовательности токенов $[A][B] \dots [A]$, предсказать следующий токен $[B]$. Высокая индукционная оценка указывает на то, что голова способна копировать информацию из удалённых позиций.

Инференс (*Inference*) — этап использования обученной модели для получения предсказаний. В контексте LLM включает предварительное заполнение KV-кэша (этап prefill) и последующую пошаговую генерацию токенов (этап decode).

KV-кэш — структура данных, в которой при авторегрессионной генерации накапливаются векторы ключей (K) и значений (V) всех ранее обработанных токенов. Объём KV-кэша растёт линейно с длиной контекста и является одним из основных ограничений инференса при длинных последовательностях.

Линейное внимание (*Linear Attention*) — разновидность self-attention, в которой экспоненциальное ядро $\exp(q^\top k)$ заменяется скалярным произведением в пространстве признаков отображений $\phi(q)^\top \phi(k)$ [2]. Такая замена допускает рекуррентное вычисление с линейной сложностью по длине последовательности.

Линеаризация внимания — в строгом смысле: замена нелинейного Softmax-ядра линейной ядерной аппроксимацией вида $\phi(q)^\top \phi(k)$, что приводит к линейной вычислительной сложности. Скользящее окно не является линеаризацией, поскольку сохраняет Softmax-ядро. В настоящей работе оба подхода объединяются под общим понятием *субквадратичного режима*. В заголовке работы термин «линеаризация» используется в расширенном значении, охватывающем все типы субквадратичных замен.

Многоголовое внимание (*Multi-Head Attention, МНА*) — схема, при которой самовнимание вычисляется параллельно в нескольких независимых подпространствах, называемых *головами*. Каждая голова имеет собственные проекции запросов (Q), ключей (K) и значений (V), что позволяет разным

головам фокусироваться на разных аспектах контекста. Результаты всех голов объединяются конкатенацией и линейной проекцией.

Передача внимания (*Attention transfer*) — процедура, при которой параметры нового субквадратичного блока обучаются воспроизводить выходы исходного Softmax-блока на тех же входных данных. Используется на первом этапе метода LoLCATs [3]. Исходные веса модели при этом не обновляются.

Перплексия (*Perplexity, PPL*) — метрика качества языковой модели, равная экспоненте средней кросс-энтропии по токенам. Чем ниже перплексия, тем лучше модель предсказывает следующий токен.

Полносвязный слой (*Multi-Layer Perceptron, MLP*) — нейросетевой блок, состоящий из двух линейных проекций с нелинейной активацией между ними. В каждом слое трансформера полносвязный слой следует за self-attention и обрабатывает каждую позицию независимо.

Потоковые головы (*Streaming heads*) — головы внимания, фокусирующиеся преимущественно на недавних токенах и токенах-аттракторах.

Предварительное заполнение (*Prefill*) — первый этап инференса, при котором модель обрабатывает весь входной текст (промпт) целиком и заполняет KV-кэш. Сложность этого этапа квадратична по длине входа для Softmax-внимания.

Признаковое отображение (*Feature map*) — функция $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$, используемая в линейном внимании для аппроксимации Softmax-ядра. Отображение переводит вектор из исходного пространства в новое, в котором скалярное произведение $\phi(q)^\top \phi(k)$ приближает $\exp(q^\top k)$.

Причинная свёртка (*Causal convolution*) — одномерная свёртка по последовательности токенов, в которой фильтр применяется только к текущей и предшествующим позициям. Слово «причинная» указывает на то, что при вычислении не используется информация из будущих позиций, что необходимо при генерации текста.

Самовнимание (*Self-Attention*) — операция, при которой каждый элемент последовательности формирует своё представление путём взвешенного агрегирования представлений всех элементов доступного контекста. В стандартной реализации весовые коэффициенты вычисляются с помощью функции Softmax.

Скрытое состояние (*Hidden state*) — промежуточное векторное пред-

ставление токена, формируемое внутри модели. На каждом слое трансформера скрытое состояние обновляется: сначала проходит через self-attention, затем через полносвязный слой. Входной токен кодируется в начальное скрытое состояние, а итоговое скрытое состояние последнего слоя используется для предсказания следующего токена.

Скольльзящее окно (*Sliding Window, SW*) — приём ограничения области self-attention, при котором каждый токен обращается только к W ближайшим предшествующим позициям. Сохраняет Softmax-ядро, но ограничивает контекст фиксированным окном, что даёт сложность $O(LW)$.

Softmax-внимание — вариант самовнимания, в котором весовые коэффициенты нормируются функцией Softmax, что позволяет формировать резкие, концентрированные распределения. Имеет квадратичную сложность $O(L^2d)$, где L — длина последовательности, d — размерность головы.

Субквадратичный режим — общее название для разновидностей self-attention со сложностью ниже $O(L^2)$: линейное внимание, скользящее окно, статические линейные замены.

Токены-аттракторы (*Attention sinks*) — небольшое число начальных токенов последовательности, получающих непропорционально высокие веса внимания от большинства голов. Это явление наблюдается эмпирически во многих LLM: даже функционально незначимые токены (например, начальный разделитель) устойчиво притягивают внимание.

Трансформер (*Transformer*) — архитектура нейронных сетей, предложенная в [1], основу обработки последовательностей в которой составляет самовнимание (self-attention).

Эмерджентная способность (*Emergent ability*) — свойство модели, которое отсутствует у её отдельных компонентов и возникает только при достаточном масштабе или сложности всей системы. В контексте LLM — способность, не заложенная явно при обучении, а проявляющаяся как побочный результат обработки данных через многослойный стек self-attention и полносвязных слоёв.

Эхо-оценка (*Echo score*) — числовая мера способности головы внимания обращаться к точной копии текущего токена в удалённой позиции последовательности. Дополняет индукционную оценку, выявляя головы с функцией буквального извлечения.

Needle-in-a-Haystack (*NIAH*) — бенчмарк, проверяющий способность модели найти и извлечь конкретный факт, размещённый в произвольной позиции длинного контекста.

1. Введение

1.1. Предметная область и актуальность

За последние несколько лет допустимая длина контекста в больших языковых моделях (LLM) выросла примерно на два порядка: от 2–4 тысяч токенов до сотен тысяч. Этот рост продиктован практическим запросом на анализ длинных документов, репозитории исходного кода и ведение глубоких многошаговых диалогов. Параллельно развивались архитектуры, изначально способные работать с длинными последовательностями.

Однако расширение контекстного окна сопряжено со значительными вычислительными затратами. Стандартное Softmax-внимание, лежащее в основе архитектуры трансформера, имеет квадратичную сложность $O(L^2d)$ относительно длины последовательности L (где d — размерность головы). На этапе инференса это создаёт два основных узких места. Время предварительного заполнения KV-кэша растёт квадратично с длиной входа. Объём самого KV-кэша на каждом слое увеличивается линейно с длиной контекста. При масштабировании до величин порядка 10^5 токенов KV-кэш одной модели начинает занимать десятки гигабайт видеопамяти [9]. В итоге именно дефицит памяти, а не интенсивность вычислений, становится главным барьером для развёртывания моделей на практике.

Один из способов обойти этот барьер — перевод части или всех слоёв внимания в субквадратичный режим. К субквадратичным подходам относятся линейное внимание [2], скользящее окно, а также модели в пространстве состояний (*State-Space Models, SSM*). Однако прямая замена Softmax-внимания субквадратичным аналогом в уже обученной модели обычно приводит к резкому падению качества. В первую очередь ухудшается способность модели точно находить и извлекать конкретные факты из длинного текста.

В данной работе исследуется компромиссный подход — гибридизация существующей модели, то есть перевод части голов внимания в субквадратичный режим при сохранении остальных в полном Softmax-режиме. Головы внимания — это независимые подпространства, в которых параллельно вычисляется self-attention (подробное определение дано в перечне терминов). Критерий разделения основан на концентрации голов извлечения: головы, отвечающие за точечное извлечение фактов из дальнего контекста, сохраняются

в полном режиме, а остальные переводятся в субквадратичный. Такой подход позволяет оптимизировать работу с контекстом без обновления весов базовой модели.

1.2. Цель, гипотеза и задачи работы

Необходимо уточнить два ключевых понятия, используемых в данной работе.

Во-первых, под *линеаризацией внимания* в строгом смысле понимается замена нелинейного Softmax-ядра линейной ядерной аппроксимацией вида $\phi(q)^\top \phi(k)$, допускающей рекуррентное вычисление (подробное определение дано в перечне терминов). Скользящее окно, при котором Softmax-ядро сохраняется, но область суммирования ограничивается фиксированным числом позиций, не является линеаризацией в этом смысле. Тем не менее оба подхода обеспечивают субквадратичную сложность по длине контекста и объединяются в данной работе под общим понятием *субквадратичного режима*. В заголовке работы термин «линеаризация» используется в расширенном значении, охватывающем оба типа субквадратичных замен.

Во-вторых, под *дообучением* (*fine-tuning*) понимается обновление весов самой предобученной модели. В отличие от дообучения, *адаптация без дообучения* сохраняет все исходные веса замороженными. Метод LoLCATs [3] вводит новые обучаемые параметры (признаковые отображения, проекции вентилей), однако исходные 4 миллиарда весов базовой модели не изменяются. По этой причине LoLCATs квалифицируется не как дообучение, а как адаптация с замороженной базой: новые параметры обучаются воспроизводить выходы исходного Softmax-внимания, но сама модель остаётся нетронутой. Конфигурация Selective Sliding Window не вводит никаких новых параметров.

Цель работы — на примере модели Qwen3-4B проверить гипотезу о преимуществе частичной гибридизации внимания. Предполагается, что подход, при котором определённая часть голов внимания сохраняется в полном Softmax-режиме на основе критерия концентрации голов извлечения, позволяет достичь лучшего баланса между качеством работы модели и её вычислительной эффективностью, чем полный перевод всех голов в субквадратичный режим.

В качестве основной гипотезы выдвинуто предположение о существо-

вании таких конфигураций частичной гибридизации без обновления весов базовой модели, которые при сопоставимой экономии KV-кэша сохраняют точность точечного извлечения информации и общее качество языкового моделирования существенно лучше, чем полная гибридизация с передачей внимания по методу LoLCATs.

Задачи работы включают последовательное выполнение нескольких этапов. Сначала проводится систематический анализ существующих методов линеаризации внимания (LoLCATs [3], LAWCAT [7], NBL [6]) и алгоритмов идентификации функционально специализированных голов внимания (RazorAttention [8], DuoAttention [9]). На основе протокола RazorAttention разрабатывается программный модуль для поиска голов извлечения в архитектуре Qwen3-4B с использованием индукционных и эхо-оценок (определения этих метрик даны в перечне терминов).

Далее на базе Qwen3-4B реализуются три гибридные архитектуры, охватывающие весь спектр подходов к частичной гибридизации голов внимания: частичная гибридизация без адаптации (Selective Sliding Window), частичная с адаптацией (Partial LoLCATs) и полная с адаптацией (Full LoLCATs). Для настройки параметров субквадратичных ветвей применяется процедура передачи внимания, соответствующая первому этапу метода LoLCATs.

В завершение работы проводится количественное сравнение четырёх полученных конфигураций по двум ключевым группам метрик: точности точечного извлечения информации в тесте NIAH и общему качеству языкового моделирования, оцениваемому через перплексию на датасете WikiText-2. На основе собранных данных проверяется исходная гипотеза и формулируются рекомендации по выбору оптимальной конфигурации.

2. Обзор литературы

2.1. Полное внимание и проблема длинного контекста

Современные LLM в большинстве случаев основаны на архитектуре трансформера, ключевым компонентом которой является self-attention [1]. Задача self-attention состоит в том, чтобы для каждого токена построить представление с учётом остальных токенов доступного контекста. Благодаря этому модель может учитывать как локальные связи между соседними словами, так и дальние зависимости, важные при анализе документов, программного кода и длинных диалогов.

Для входной последовательности $X \in \mathbb{R}^{L \times d_{\text{model}}}$, где L — длина контекста, а d_{model} — размерность скрытого представления, в каждой голове внимания вычисляются запросы, ключи и значения:

$$Q = XW_q, \quad K = XW_k, \quad V = XW_v. \quad (1)$$

В авторегрессионной модели (то есть модели, генерирующей токены последовательно, по одному) с причинным маскированием выход для позиции n имеет вид:

$$y_n = \sum_{i=1}^n \frac{\exp(q_n^\top k_i / \sqrt{d})}{\sum_{j=1}^n \exp(q_n^\top k_j / \sqrt{d})} v_i. \quad (2)$$

Преимущество такого подхода состоит в высокой выразительности: Softmax-нормировка позволяет модели концентрировать внимание на небольшом числе наиболее релевантных токенов. Однако это приводит к двум практическим ограничениям. Во-первых, на этапе предварительного заполнения KV-кэша необходимо вычислять попарные взаимодействия между токенами, что даёт сложность $O(L^2 d)$. Во-вторых, при генерации модель хранит ключи и значения всех предыдущих токенов, поэтому объём KV-кэша растёт линейно с длиной контекста. При контекстах порядка сотен тысяч или миллионов токенов именно KV-кэш становится одним из главных ограничений инференса. В работе [9] отмечается, что для модели Llama-3-8B хранение KV-кэша в формате FP16 (16-битное представление с плавающей точкой) на 1 млн токенов требует не менее 137 ГБ видеопамяти, что превышает объём одного ускорителя с 80 ГБ.

Для кодирования позиций в последовательности используются различные подходы: абсолютные синусоидальные кодировки [1], аддитивные позиционные смещения ALiBi [11] и вращательные позиционные кодировки RoPE [12]. Выбор позиционного кодирования влияет на способность модели экстраполировать на длины, превышающие те, на которых она обучалась. Модель Qwen3-4B, исследуемая в данной работе, использует RoPE.

Следовательно, при переходе к длинному контексту возникает противоречие: полное Softmax-внимание обеспечивает высокое качество извлечения информации, но плохо масштабируется по времени и памяти. Это противоречие лежит в основе большинства современных работ по ускорению инференса и переводу слоёв внимания в субквадратичный режим.

2.2. Линейное внимание и его ограничения

Один из способов снизить вычислительную стоимость self-attention — заменить Softmax-ядро линейным аналогом. В этом подходе экспоненциальное ядро аппроксимируется скалярным произведением после применения признакового отображения:

$$\exp(q^\top k) \approx \phi(q)^\top \phi(k), \quad (3)$$

где ϕ — признаковое отображение (функция, переводящая вектор из исходного пространства в новое, в котором скалярное произведение приближает экспоненциальное ядро). Тогда выход внимания можно записать как:

$$\hat{y}_n = \frac{\phi(q_n)^\top \left(\sum_{i=1}^n \phi(k_i) v_i^\top \right)}{\phi(q_n)^\top \left(\sum_{j=1}^n \phi(k_j) \right)}. \quad (4)$$

Суммы в числителе и знаменателе можно обновлять рекуррентно:

$$S_n = S_{n-1} + \phi(k_n) v_n^\top, \quad z_n = z_{n-1} + \phi(k_n), \quad (5)$$

где $S_n \in \mathbb{R}^{d' \times d}$ — матрица рекуррентного состояния, $z_n \in \mathbb{R}^{d'}$ — нормировочный вектор.

В результате модель хранит не полный KV-кэш размера $O(Ld)$, а компактное состояние S_n и нормировочный вектор z_n фиксированной размерности.

Это позволяет снизить сложность инференса с $O(L^2)$ до $O(L)$ по длине последовательности и получить постоянный объём памяти при авторегрессионной генерации [2]. Однако на практике такая замена часто ухудшает качество модели. Причина состоит в том, что Softmax-внимание формирует резкие, концентрированные распределения, тогда как линейные аппроксимации склонны к более размытому усреднению контекста. Для задач точечного извлечения информации, например NIAH, это особенно критично: модель должна найти конкретный фрагмент в длинной последовательности, а не усреднить информацию по всему тексту.

2.3. Post-hoc линейаризация: LoLCATs, LAWCAT и Lizard

Термин *post-hoc* (лат. «после свершившегося») означает применение модификации к уже обученной модели, в противоположность обучению новой архитектуры с нуля. Поскольку обучение крупных моделей с линейным вниманием с нуля требует значительных вычислительных ресурсов, отдельное направление исследований связано с post-hoc линейаризацией уже обученных моделей на основе трансформера. Под post-hoc линейаризацией понимается замена стандартного self-attention в готовой модели на субквадратичный аналог без полного предобучения новой архитектуры.

Одним из основных подходов в этом направлении является LoLCATs [3]. Метод состоит из двух этапов. На первом этапе выполняется передача внимания: обучаемые субквадратичные аналоги подбираются так, чтобы их выходы приближали выходы исходного Softmax-внимания по среднеквадратичной ошибке (MSE). На втором этапе применяется низкоранговая адаптация LoRA [4] — метод, при котором обновление весов представляется в виде произведения двух низкоранговых матриц, что существенно сокращает число обучаемых параметров. LoRA компенсирует остаточные ошибки аппроксимации. Важная особенность LoLCATs — использование гибридной схемы: локальный контекст обрабатывается точным Softmax-вниманием в скользящем окне, а более дальний контекст — линейным вниманием. Такой подход позволяет лучше сохранить локальную точность, чем полная замена self-attention.

Метод LoLCATs демонстрирует, что линейаризация готовых моделей может быть существенно эффективнее обучения с нуля. В работе [3] отмеча-

ется, что метод использует около 40 млн токенов и обучает лишь малую долю параметров по сравнению с предыдущими подходами. Метод масштабируется до моделей с 70 и 405 миллиардами параметров. Однако для настоящей работы важны и ограничения этого подхода. Во-первых, LoLCATs требует адаптации новых параметров (хотя базовые веса остаются замороженными). Во-вторых, метод не задаёт строгого функционального критерия, какие слои следует оставлять в полном режиме self-attention. Выбор сохраняемой части архитектуры остаётся эвристическим.

LAWCAT [7] развивает идею передачи знаний из квадратичного self-attention в линейное. Его отличие состоит в добавлении причинной свёртки Conv1D по токенам (одномерной свёртки, использующей только текущую и предшествующие позиции) и нормированного управляемого линейного внимания. Свёртка усиливает моделирование локальных зависимостей, а вентиль забывания помогает управлять накоплением информации в рекуррентном состоянии. По данным авторов, вариант LAWCAT на базе Mistral-7B, обученный на последовательностях длиной около 1К токенов, сохраняет более 90% точности на задаче извлечения ключа из контекста (*passkey retrieval*) вплоть до 22К токенов [7]. В этой задаче модель должна найти заранее заданную строку в длинном тексте — по своей сути она близка к NIAH. Тем не менее LAWCAT также требует адаптации новых параметров, поэтому не является методом без дообучения.

Близкую мотивацию имеет Lizard [16]. Этот метод заменяет стандартное self-attention гибридной архитектурой, сочетающей управляемое линейное внимание для глобального сжатия контекста и локальное якорное окно (*Anchor Window Attention*) для сохранения точности ближайших взаимодействий. В отличие от более жёстких линеаризаций, Lizard вводит обучаемые модули управления памятью, что позволяет модели адаптивно решать, какую информацию сохранять, а какую забывать. В работе [16] также подчёркивается, что гибридная конфигурация с сохранением части исходных Softmax-слоёв практически достигает качества учительской модели. Это подтверждает общий вывод: полная замена self-attention менее эффективна, чем гибридизация.

2.4. Головы извлечения и сжатие KV-кэша

Другой класс методов не заменяет self-attention субквадратичным аналогом, а сокращает KV-кэш. Важное наблюдение состоит в том, что не все головы внимания одинаково используют длинный контекст. В работах [8] и [9] показано, что внутри модели можно выделить два типа голов: головы извлечения и потоковые головы.

Головы извлечения способны обращаться к удалённым, но релевантным токенам. Они играют ключевую роль в задачах, где требуется найти конкретный факт в длинном тексте. Потоковые головы в основном фокусируются на недавних токенах и токенах-аттракторах (начальных токенах последовательности, устойчиво притягивающих высокие веса внимания), поэтому не нуждаются в полном доступе ко всему контексту. Это различие позволяет применять разные стратегии хранения KV-кэша для разных голов.

RazorAttention [8] предлагает метод сжатия KV-кэша, не требующий дообучения. Для голов извлечения сохраняется полный кэш, а для потоковых голов удаляются записи о далёких токенах. Чтобы частично компенсировать потерю удалённой информации, для потоковых голов используется специальный *компенсационный токен* — дополнительная запись в KV-кэше, которая аккумулирует обобщённое представление удалённого контекста вместо полного набора удалённых ключей и значений. Авторы показывают, что такой подход позволяет сократить KV-кэш более чем на 70% без заметного ухудшения качества на задачах с длинным контекстом. Важный вывод RazorAttention состоит в том, что критическая информация не обязательно равномерно распределена по всем головам: часто только малая часть голов отвечает за извлечение из дальнего контекста.

DuoAttention [9] формализует это разделение через обучаемый вентиль для каждой головы. На этапе идентификации вводится параметр $\alpha_{i,j}$, который смешивает полное и потоковое внимание:

$$\text{attn}_{i,j} = \alpha_{i,j} \text{full_attn} + (1 - \alpha_{i,j}) \text{streaming_attn}. \quad (6)$$

Если после оптимизации $\alpha_{i,j}$ принимает значение, близкое к единице, голова считается головой извлечения и получает полный KV-кэш. В противном случае она работает в потоковом режиме с кэшем постоянной длины. При

этом параметры $\alpha_{i,j}$ оптимизируются при замороженных весах модели, что делает метод адаптацией, а не дообучением. Авторы сообщают о снижении потребления памяти в 2,55 раза для моделей с архитектурой MHA и в 1,67 раза для моделей с архитектурой GQA при минимальной потере точности [9].

Оба метода важны для настоящего исследования, поскольку они показывают функциональную неоднородность self-attention. Однако RazorAttention и DuoAttention работают на уровне отдельных голов внутри слоя. Такая схема точна, но усложняет реализацию: внутри одного слоя необходимо одновременно поддерживать разные режимы обработки кэша.

2.5. Замена блоков без дообучения: NBL

Метод Neural Block Linearization (NBL) [6] предлагает иной подход к ускорению инференса. В нём отдельные слои self-attention заменяются статическим линейным преобразованием:

$$\hat{Y} = W_{\text{lin}}X + b, \quad (7)$$

где W_{lin} — матрица полной линейной аппроксимации слоя (не следует путать с матрицами проекций W_q , W_k , W_v из формулы (1)). Параметры W_{lin} и b вычисляются аналитически с помощью линейной оценки минимальной среднеквадратичной ошибки (*LMMSE* — метод, при котором линейное отображение подбирается так, чтобы минимизировать среднеквадратичное отклонение выхода от эталона). Выбор слоёв для замены выполняется на основе канонического корреляционного анализа (*CCA* — статистический метод, измеряющий степень линейной зависимости между двумя наборами переменных). Если выход слоя хорошо предсказывается линейной функцией его входа, такой слой считается подходящим для замены.

Преимущество NBL состоит в том, что метод не требует градиентного обучения. В экспериментах авторы показывают, что замена 12 слоёв self-attention в модели DeepSeek-R1-Distill-Llama-8B увеличивает скорость инференса на 32% при потере точности менее 1% [6]. Однако у этого подхода есть принципиальное ограничение: статическое линейное преобразование не обладает контекстной адаптивностью, свойственной self-attention. Поэтому NBL хорошо подходит для замены избыточных слоёв, но может быть

недостаточным для слоёв, отвечающих за точечное извлечение информации.

2.6. Выводы для настоящей работы

Рассмотренные исследования показывают, что задача эффективной работы с длинным контекстом не сводится к простой замене полного self-attention субквадратичным аналогом. Полное Softmax-внимание затратно по памяти и времени, но именно оно лучше всего сохраняет способность модели к точному извлечению информации. Субквадратичные подходы снижают вычислительную сложность, однако часто страдают от размывания контекста и хуже справляются с задачами поиска конкретного факта.

Методы LoLCATs, LAWCAT и Lizard показывают, что гибридные схемы обычно устойчивее полной линейаризации. Они сохраняют часть точного self-attention и заменяют оставшуюся часть более эффективными подходами. Однако такие методы либо требуют адаптации дополнительных параметров, либо не дают строгого критерия, какие именно слои следует сохранять в полном режиме.

RazorAttention и DuoAttention, в свою очередь, показывают, что способность к извлечению информации из дальнего контекста сосредоточена не во всей модели равномерно, а в ограниченном числе специализированных голов. Из этого следует важная идея для настоящей работы: при гибридизации следует учитывать не только номер слоя, но и концентрацию в нём голов извлечения.

На этом основании в данной работе исследуется частичная гибридизация модели Qwen3-4B. KV-группы, содержащие хотя бы одну голову извлечения, сохраняются в полном Softmax-режиме; остальные KV-группы переводятся в субквадратичный режим. Такой подход отличается от LoLCATs тем, что выбор защищаемых голов задаётся функциональным критерием (концентрация голов извлечения), а не фиксированной эвристикой. От RazorAttention и DuoAttention он отличается типом вмешательства: те методы сжимают KV-кэш потоковых голов, удаляя далёкие записи и сохраняя Softmax-ядро, тогда как данная работа полностью заменяет режим вычислений потоковых голов на субквадратичный (скользящее окно или линейное внимание). Гранулярность при этом одна — KV-группа внутри слоя, что совместимо с архитектурой GQA и потенциально упрощает реализацию в системах инференса (специализиро-

ванном программном обеспечении для выполнения моделей, например vLLM). От NBL отличие состоит в том, что критерий замены связан не с линейной аппроксимируемостью слоя, а с функциональной ролью голов внимания в извлечении информации из дальнего контекста.

3. Предлагаемый метод

3.1. Концепция частичной гибридизации

Схема адаптации произвольной предобученной LLM состоит из трёх этапов.

1. **Идентификация голов извлечения** по протоколу RazorAttention [8]: для каждой головы вычисляются эхо-оценка и индукционная оценка на синтетической последовательности.
2. **Построение бинарной маски** голов извлечения с заданным порогом отсечения p (доля защищённых Q -голов). Для совместимости с архитектурой GQA (групповое внимание, в котором k голов запросов совместно используют одну пару ключ–значение; такая группа называется *KV-группой*) статус удержания распространяется на уровень целой KV-группы по правилу логического ИЛИ: если хотя бы одна из k Q -голов группы помечена как голова извлечения, вся группа считается защищённой.
3. **Применение маски** как селектора режима на каждом слое: защищённые KV-группы вычисляются в полном Softmax-режиме и сохраняют весь свой KV-кэш; остальные группы переводятся в субквадратичный режим — скользящее окно $W = 512$ с 4 токенами-аттракторами (Selective SW) либо гибрид скользящего окна и управляемого линейного внимания GLA [10] (Partial и Full LoLCATs).

Важной архитектурной особенностью такого подхода является нелинейный рост объёма защищённых KV-групп с долей голов извлечения. В предположении независимого выбора Q -голов доля защищённых KV-групп оценивается как $1 - (1 - p)^k$. Смысл этой формулы: если каждая из k голов в группе с вероятностью p является головой извлечения, то вероятность того, что *ни одна* голова в группе не помечена, равна $(1 - p)^k$, а дополнение $1 - (1 - p)^k$ даёт долю групп, в которых есть хотя бы одна голова извлечения. Для Qwen3-4B ($k = 4$) при $p = 0,15$ это даёт около 48%, то есть примерно вдвое меньшую долю полных KV-кэшей по сравнению с исходной моделью. На практике выбор Q -голов не является независимым (см. визуализацию масок

на рис. 1 — концентрация в средних и поздних слоях), поэтому величина $1 - (1 - p)^k$ является приближённой оценкой для случайного распределения голов, а не универсальной верхней границей. Для точного расчёта экономии памяти нужно считать не долю защищённых Q -голов, а долю уникальных защищённых KV-групп, полученную после применения правила ИЛИ внутри каждой GQA-группы. Дополнительно незащищённые группы в режиме Selective SW сохраняют усечённый кэш размера $K = W + S = 516$ позиций, что при длинном контексте ($L \gg W$) даёт добавку к памяти на порядок меньшую, чем полный KV-кэш.

Верхняя оценка объёма KV-кэша. Рассмотрим идеальную реализацию, в которой полный KV-кэш и укороченный кэш для скользящего окна физически хранятся отдельно. Пусть L — длина контекста, W — размер скользящего окна, S — число токенов-аттракторов, а $K = W + S$ — число позиций, сохраняемых для незащищённых KV-групп. В данной работе $W = 512$, $S = 4$, поэтому $K = 516$.

В архитектуре Qwen3-4B используется GQA: каждая KV-группа обслуживает $k = 4$ Q -головы. Поэтому доля защищённых Q -голов не совпадает напрямую с долей защищённых KV-групп. Если защищена доля p_q всех Q -голов, то в наиболее неблагоприятном случае каждая защищённая Q -голова может попасть в отдельную KV-группу. Тогда верхняя граница доли защищённых KV-групп равна:

$$p_{kv}^{\max} = \min(1, kp_q). \quad (8)$$

Эта оценка является верхней границей: она показывает, какой максимальный объём KV-кэша может потребоваться после переноса маски с уровня Q -голов на уровень KV-групп. Реальный объём может быть меньше, если несколько защищённых Q -голов попадают в одну и ту же KV-группу.

Для модели Qwen3-4B всего имеется $36 \times 32 = 1152$ Q -голов и $36 \times 8 = 288$ KV-групп. Для маски default, где защищено около 15% Q -голов, число защищённых Q -голов можно оценить сверху как:

$$n_q^{\max} = \lceil 0,15 \cdot 1152 \rceil = 173. \quad (9)$$

В худшем случае эти 173 головы могут затронуть 173 разные KV-группы.

Следовательно, верхняя граница доли защищённых KV-групп для маски default равна:

$$p_{kv}^{\max} = \frac{173}{288} \approx 0,601. \quad (10)$$

Иными словами, даже при наиболее невыгодном распределении защищённых голов не более 60,1% KV-групп должны хранить полный KV-кэш. Остальные KV-группы в идеальной реализации Selective SW хранят только локальное окно и токены-аттракторы.

Если доля защищённых KV-групп равна $p_{kv}^{\max}$, то для них необходимо хранить полный кэш длины L . Для остальных $(1 - p_{kv}^{\max})$ групп достаточно хранить только $K = W + S$ позиций. Поэтому верхняя оценка относительного объёма KV-кэша по сравнению с исходной моделью имеет вид:

$$R_{KV}^{\max} = p_{kv}^{\max} + (1 - p_{kv}^{\max}) \frac{K}{L}. \quad (11)$$

Первое слагаемое соответствует защищённым KV-группам, которые продолжают работать в полном Softmax-режиме. Второе слагаемое соответствует незащищённым группам, для которых хранится только локальное окно и несколько начальных токенов-аттракторов. При росте длины контекста вклад второго слагаемого уменьшается, а верхняя оценка памяти стремится к $p_{kv}^{\max}$.

Таблица 1: Верхняя оценка доли защищённых KV-групп после переноса маски с Q -голов на KV-группы при GQA ($k = 4$).

Маска	Защищённые Q -головы p_q	Формула	$p_{kv}^{\max}$
default	$\sim 15\%$	$173/288$	$\sim 60,1\%$
medium	$\sim 32\%$	$\min(1, 4 \cdot 0,32)$	100%
safe	$\sim 50\%$	$\min(1, 4 \cdot 0,50)$	100%

Из табл. 1 видно, что для маски default даже в худшем случае защищено не более 60,1% KV-групп. Для масок medium и safe верхняя граница равна 100%, поскольку при неудачном распределении защищённые Q -головы могут затронуть все KV-группы. Это не означает, что фактическая экономия памяти обязательно отсутствует; это означает только, что по одной верхней оценке для этих масок нельзя гарантировать сокращение KV-кэша.

Для маски default при длине контекста $L = 32768$ получаем:

$$R_{KV}^{\max} = \frac{173}{288} + \left(1 - \frac{173}{288}\right) \frac{516}{32768} \approx 0,607. \quad (12)$$

Таблица 2: Верхняя оценка относительного объёма KV-кэша в идеальной реализации Selective SW при $W = 512$, $S = 4$, $K = 516$.

Маска	$p_{kv}^{\max}$	$L = 4K$	$L = 8K$	$L = 32K$
default	$\sim 60,1\%$	65,1%	62,6%	60,7%
medium	100%	100%	100%	100%
safe	100%	100%	100%	100%

Следовательно, в идеальной реализации Selective SW максимальный объём KV-кэша для маски default на длине 32K составляет около 60,7% от исходной модели. Иными словами, даже в худшем случае ожидается сокращение KV-кэша примерно в:

$$\frac{1}{0,607} \approx 1,65 \quad (13)$$

раза.

3.2. Исследуемые конфигурации

В финальном сравнении участвуют четыре конфигурации:

1. **Original Qwen3** — исходная полноразмерная модель, эталон качества;
2. **Selective Sliding Window** (Selective SW) — 15% голов извлечения остаются в полном Softmax-внимании, остальные переведены в скользящее окно $W = 512$; адаптация не требуется;
3. **Partial LoLCATs** — те же 15% голов извлечения сохранены, остальные дополнительно получают линейную GLA-ветвь; требуется передача внимания (Stage-1);
4. **Full LoLCATs** — 0% голов извлечения, все 1152 головы переведены в гибридный режим; требуется передача внимания (Stage-1).

4. Экспериментальная установка

4.1. Базовая модель и окружение

Объект исследования — модель Qwen/Qwen3-4B [13] в точности BFloat16 (16-битное представление с плавающей точкой, использующее расширенный динамический диапазон). Модель содержит 36 трансформерных блоков с 32 Q -головами и 8 KV-группами (архитектура GQA, то есть каждые 4 головы запросов совместно используют одну пару ключ–значение) на слой, что в сумме составляет $36 \times 32 = 1152$ Q -голов и $36 \times 8 = 288$ KV-групп. Для кодирования позиций используются вращательные позиционные кодировки RoPE [12]. Размер модели выбран так, чтобы инференс на 32K токенов помещался в 24 ГБ видеопамати одной NVIDIA RTX 3090. Второй мотив — архитектурное соответствие: в семействе Qwen3 уже используется чередование Softmax- и линейных слоёв в пропорции 1:3 в моделях, обученных с нуля. В данной работе проверяется, можно ли получить аналогичную схему в режиме адаптации без дообучения.

Окружение для всех экспериментов единое:

- GPU: NVIDIA RTX 3090, 24 ГБ видеопамати; драйвер 580.105.08; CUDA 13.0;
- инференс: vLLM — высокопроизводительная система инференса LLM с открытым исходным кодом, оптимизирующая планирование запросов и управление памятью GPU (используется для замера пропускной способности исходной модели); HuggingFace Transformers — популярная библиотека для работы с предобученными нейросетевыми моделями, предоставляющая унифицированный интерфейс загрузки и инференса (используется для NIAH и идентификации голов); совместно с FlashAttention-2 — оптимизированным вычислительным ядром (GPU kernel) для Softmax-внимания, снижающим потребление памяти за счёт разбиения матриц на блоки (тайлинг) и переупорядочивания операций;
- линейная ветвь гибрида (GLA): библиотека flash-linear-attention — набор оптимизированных вычислительных ядер для линейного внимания; используются ядро chunk_gla (обработка последовательности блоками на этапе предварительного заполнения) и fused_recurrent_gla

(рекуррентный пошаговый режим на этапе декодирования).

4.2. Программная реализация

Программная часть написана на Python и разбита на три модуля.

Модуль идентификации голов извлечения

Модуль `code/src/razor_attn` реализует протокол RazorAttention [8] с поддержкой GQA. Компоненты:

- `run_ra.py` — прогоняет синтетическую последовательность (2500 случайных токенов, повторённых 4 раза, итого около 10K) через полноразмерную модель и сохраняет матрицы внимания для каждой головы;
- `ra_rope_tools.py` — вычисляет эхо-оценку и индукционную оценку для каждой головы, отбирает $\text{top-}p$ голов по каждому критерию;
- `save_masks.py` — сохраняет три бинарные маски формы (1152,) и распространяет статус головы извлечения на уровень целой KV-группы по правилу ИЛИ (для совместимости с GQA);
- `qwen3_sliding_window.py` — модифицированный модуль self-attention для Qwen3, который принимает маску голов: отмеченные головы вычисляют полное Softmax-внимание, остальные — используют скользящее окно $W = 512$ с 4 токенами-аттракторами.

Модуль гибридного слоя LoLCATs

Модуль `code/src/loLCats` реализует гибридную ветвь, сочетающую скользящее окно и управляемое линейное внимание GLA [10] по схеме LoLCATs [3]:

$$\hat{y} = \alpha \cdot \text{SW}(Q, K, V) + (1 - \alpha) \cdot \text{GLA}(\phi_q(Q), \phi_k(K), V, g), \quad (14)$$

где α — обучаемый скаляр смешивания (определяет баланс между скользящим окном и линейной ветвью), ϕ_q и ϕ_k — признаковые отображения (функции, аппроксимирующие Softmax-ядро), g — вектор вентилей забывания (управляет тем, какую долю рекуррентного состояния GLA сохраняет на каждом шаге).

На каждый слой добавляются обучаемые параметры:

- `phi_q.weight`, `phi_k.weight` формы $(H, d_{\text{head}}, d_{\text{head}})$ — признаковые отображения для каждой головы в стиле метода Hedgehog [5]; реализация упрощённая по сравнению с оригиналом: $\phi(x) = \text{Softmax}(W_h x)$ без конкатенации положительной и отрицательной ветвей. Инициализация — единичная матрица ($H = 32, d_{\text{head}} = 128$);
- `g_proj` — линейная проекция `nn.Linear(hidden_size, H · dhead)`, формирующая вентили забывания для GLA через функцию $\log \sigma$; веса инициализированы нулями, смещения — нулями ($\sigma(0) = 0,5$, то есть при инициализации GLA сохраняет половину накопленного состояния на каждом шаге);
- `mix_logit` формы $(H,)$ — скаляр смешивания для каждой головы, инициализация значением 5, что даёт $\alpha \approx 0,993$, то есть в начале обучения слой практически эквивалентен чистому скользящему окну.

Основной вклад в число параметров даёт полноразмерная проекция `g_proj` (на слой: $2560 \times 4096 \approx 10,5\text{М}$ весов). С учётом признаковых отображений на слой приходится около 11,5М, на все 36 слоёв — около 415М обучаемых параметров против 4В замороженных весов базовой модели. Линейная GLA-ветвь использует вычислительные ядра `chunk_gla` и `fused_recurrent_gla` из библиотеки `flash-linear-attention`.

Передача внимания (Stage-1)

Модуль `train_lolcats.py` реализует передачу внимания по схеме LoLCATs. Минимизируется среднеквадратичная ошибка (MSE) между выходом гибридного блока (перед проекцией `o_proj`) и выходом оригинального Softmax-блока на том же входе. Если обозначить через $\text{out}_\ell^{(\cdot)} \in \mathbb{R}^{B \times T \times H \times d}$ выходы ℓ -го слоя для размера пакета B и длины последовательности T , то целевая функция — средняя поэлементная квадратичная ошибка по всем слоям:

$$\mathcal{L} = \frac{1}{36} \sum_{\ell=1}^{36} \frac{1}{BTHd} \|\text{out}_\ell^{\text{patched}} - \text{out}_\ell^{\text{original}}\|_F^2. \quad (15)$$

Особенность реализации — принудительное задание входов на уровне слоёв, называемое *teacher forcing*. Суть приёма: каждый слой ℓ обучается не на выходах предыдущего (уже модифицированного) слоя, а на скрытых

состояниях (промежуточных векторных представлениях токенов) из прямого прохода оригинальной модели при замороженных градиентах. Без этого ошибка накапливается через стек из 36 слоёв и оптимизация теряет стабильность.

Конфигурация обучения. Корпус Paul Graham essays (около 150К токенов), длина последовательности 1024, размер пакета 1, 2000 шагов, оптимизатор AdamW (вариант Adam с коррекцией регуляризации весов) с начальной скоростью обучения $3 \cdot 10^{-4}$ и разогревом. Все веса базовой модели заморожены, обучаются только новые 415М параметров.

4.3. Идентификация голов извлечения и построение масок

Алгоритм RazorAttention с тремя порогами отсечения (табл. 3) даёт три бинарные маски формы (36×32) , показанные на рис. 1.

Таблица 3: Конфигурации масок голов извлечения. Последняя колонка показывает ожидаемую долю защищённых KV-групп при предположении независимого распределения защищённых Q-голов по GQA-группам. Это не является фактическим измерением и не является верхней границей.

Маска	top induction	top echo	защищ. Q-головы	ожд. доля защищ. KV-групп
default	14%	1%	~15%	~48%
medium	30%	2%	~32%	~78%
safe	46%	4%	~50%	~93%

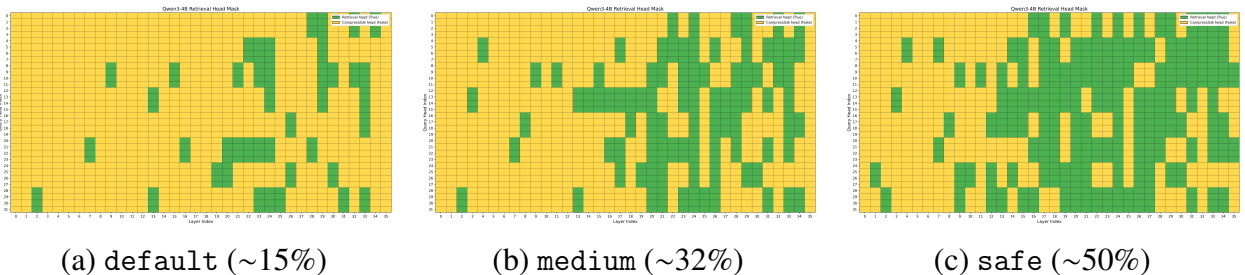


Рис. 1: Маски голов извлечения Qwen3-4B для трёх конфигураций. Зелёный — головы извлечения (полное внимание), жёлтый — потоковые (субквадратичный режим). С ростом бюджета (a → b → c) насыщаются прежде всего средние и поздние слои стека; ранние слои (0–8) во всех трёх конфигурациях почти полностью остаются потоковыми.

Качественные наблюдения по маскам. Распределение голов извлечения по слоям сильно неоднородно. Большая их часть сосредоточена в средней и поздней частях стека; ранние слои (0–8) почти целиком являются потоковыми. Это согласуется с общей картиной для LLM, обученных с нуля: ранние

слои обрабатывают локальные зависимости на уровне токенов, долгосрочное копирование и индукция реализуются глубже. С ростом бюджета (от `default` к `medium` и далее к `safe`) насыщаются прежде всего те же слои, где головы извлечения уже были выделены; в ранние слои функция извлечения не проникает. Это свидетельствует об устойчивости отбора по структуре, а не о его случайности.

4.4. Протоколы оценки

Оценка проводится по трём независимым осям. Первая ось — профиль пропускной способности исходной модели, который показывает, как полное Softmax-внимание масштабируется при росте длины контекста. Эта часть нужна не для оценки качества генерируемого текста, а для фиксации вычислительного ограничения исходной модели и последующей теоретической оценки возможного выигрыша от частичной гибридизации.

Две остальные оси относятся к качеству модели. Первая из них — точечное извлечение информации в бенчмарке `Needle-in-a-Haystack`, то есть проверка способности модели найти конкретный факт в длинном контексте. Вторая — общее качество языкового моделирования, измеряемое через перплексию на `WikiText-2`. Такое разделение позволяет отдельно анализировать эффективность инференса и сохранение функциональных возможностей модели.

Профиль пропускной способности

Чтобы зафиксировать масштаб деградации полного Softmax-внимания с ростом контекста и мотивировать переход к гибридным схемам, для исходной `Qwen3-4B` снят профиль пропускной способности через бенчмарк `vLLM` на синтетических случайных промптах. Для каждой пары (L_{in}, L_{out}) , $L_{in} \in \{4K, 8K, 16K, 32K\}$, $L_{out} \in \{128, 512, 2048\}$, запускаются 64 запроса; бенчмарк прогоняется не менее трёх раз, в отчёт идёт медиана. Замеряются скорость декодирования (токенов в секунду), общая пропускная способность, число запросов в секунду и пиковое потребление видеопамяти. Параметры декодирования зафиксированы (`temperature = 1.0`, `top_p = 1.0`, `repetition_penalty = 1.0`).

Замер пропускной способности выполнялся только для исходной модели. Его цель состоит не в прямом сравнении всех исследуемых конфигураций, а

в фиксации базового вычислительного профиля полного Softmax-внимания при росте длины контекста. Сравнительный замер скорости для гибридных конфигураций требует отдельной оптимизированной реализации вычислительных ядер скользящего окна, GLA и отдельного хранения KV-кэша для защищённых и потоковых KV-групп. Такая инженерная оптимизация выходит за рамки настоящей работы. Поэтому в дальнейшем экспериментально сравнивается качество моделей, а потенциальный выигрыш по памяти и скорости рассматривается на уровне теоретической оценки.

Точечное извлечение: Needle-in-a-Haystack

Бенчмарк NIAN проверяет, может ли модель извлечь конкретный факт, размещённый в произвольной позиции длинного контекста. В корпус Paul Graham essays (набор эссе общим объёмом порядка сотен тысяч токенов) вставляется строка-«игла»

"The secret code is: XJ9-ALPHA-7428-BETA",

а в конец контекста — вопрос "What is the secret code?". Ответ считается правильным, только если в нём есть точная подстрока с кодом.

В протоколе варьируются два параметра:

- длина контекста $L_{in} \in \{4K, 8K, 16K, 32K\}$ — позволяет наблюдать, как деградирует качество с ростом дистанции от иглы до запроса;
- относительная позиция иглы $depth_rel \in \{0; 0,5; 1\}$ (начало, середина, конец). Точность существенно зависит от этого параметра: проще всего извлечь иглу около конца контекста (короткая дистанция) или в самом начале (типичная позиция токенов-аттракторов); середина — наиболее сложный случай.

Каждая ячейка $(L_{in}, depth_rel)$ оценивается по трём прогонам с разными начальными состояниями генератора случайных чисел (*seed*); ячейка считается успешной (100%), только если точная подстрока присутствует во всех трёх ответах.

Отличие NIAN от обычного поиска. NIAN — не поиск подстроки и не индексный поиск по контексту. У языковой модели нет явной структуры

данных, в которой лежит входной текст в literalном виде. Для понимания этого различия полезно сравнить три типа извлечения информации:

- **Поиск подстроки или регулярного выражения** — детерминированный алгоритм за $O(N)$ операций по literalному тексту. Если подстрока есть, она будет найдена с гарантией; качество не зависит ни от длины, ни от позиции.
- **Векторный или индексный поиск** (BM25, плотный поиск по эмбедингам) — работает с явными индексами или векторными представлениями и выполняет приближённое извлечение за $O(\log N)$ или $O(N)$ операций над специально построенной структурой данных.
- **Контекстное извлечение LLM (NIAH)** — эмерджентная способность, то есть свойство, возникающее из совокупной работы всех компонентов модели, а не заложенное явно. У модели нет ни таблицы соответствий «токен → позиция», ни адресации по смещению. Информация об игле кодируется в скрытых состояниях (промежуточных векторных представлениях токенов) и проходит через тот же стек слоёв self-attention и полносвязных слоёв (MLP — нейросетевой блок из двух линейных проекций с нелинейной активацией, обрабатывающий каждую позицию независимо), прежде чем превратиться в финальный ответ.

NIAH-точность измеряет, способна ли внутренняя вычислительная схема модели построить маршрут от позиции иглы до позиции запроса через цепочку слоёв self-attention. Провал NIAH означает не нехватку контекста как такового, а структурную неспособность стека сохранить информационный сигнал на требуемой дистанции.

Замечание о состоянии исследований памяти LLM. То, каким образом LLM удерживает и извлекает факты из длинного контекста, до сих пор остаётся областью активных исследований. На уровне гипотез выделяют специализированные индукционные головы [14], которые формируют цепи контекстного обучения $[A][B] \dots [A] \rightarrow [B]$, и более широкий класс голов извлечения [15], эмпирически связанных с фактологичностью на длинных контекстах. Целостной картины при этом нет. Косвенный признак — разные критерии

идентификации голов извлечения (RazorAttention [8], DuoAttention [9]) дают разные наборы значимых голов. В данной работе NIAH используется как поведенческий зонд конкретной функциональной способности модели, а не как замена обычному поиску.

Качество языкового моделирования: перплексия на WikiText-2

Перплексия — экспонента средней кросс-энтропии по токенам:

$$\text{PPL} = \exp\left(-\frac{1}{N} \sum_{n=1}^N \log p(x_n | x_{<n})\right), \quad (16)$$

где N — число токенов в оценочном тексте. Метрика измеряет, насколько хорошо модель предсказывает следующий токен в естественном тексте. Она не привязана ни к извлечению фактов, ни к рассуждению, ни к коду и поэтому служит общим индикатором того, что качество языкового моделирования при архитектурной модификации не деградировало.

Замеры проводятся на тестовой части WikiText-2 при длинах последовательности $\text{seq_len} \in \{512, 2048\}$. WikiText-2 выбран сознательно: он не использовался ни на одном этапе адаптации (идентификация голов извлечения выполнялась на синтетических токенах, передача внимания — на корпусе Paul Graham essays), и поэтому даёт независимую оценку вне распределения данных адаптации. Сравнение перплексии при $\text{seq_len} = 512$ (вмещается в скользящее окно $W = 512$) и $\text{seq_len} = 2048$ (выходит за окно) позволяет отдельно оценить влияние перевода голов в субквадратичный режим на моделирование дальнего контекста: при $\text{seq_len} = W$ гибридные конфигурации формально неотличимы от полной модели, и расхождение появляется только при выходе за окно.

5. Результаты и обсуждение

5.1. Базовый профиль пропускной способности

Результаты замера пропускной способности исходной модели приведены в табл. 4.

Таблица 4: Пропускная способность Qwen3-4B (полное внимание) на vLLM, медиана по трём прогонам.

L_{in}	L_{out}	decode tok/s	total tok/s	req/s	peak mem (ГБ)	OOM
4K	128	692,1	6228,6	5,41	22,60	0
4K	2048	689,4	6204,3	5,39	22,60	0
8K	128	629,1	5661,9	4,91	22,64	0
8K	2048	628,4	5655,9	4,91	22,60	0
16K	128	503,8	4533,9	3,94	22,64	0
16K	2048	504,1	4536,9	3,94	22,60	0
32K	128	366,5	3298,5	2,86	22,30	0
32K	2048	371,0	3339,2	2,90	22,34	0

По табл. 4 видно, что при росте контекста с 4K до 32K скорость декодирования падает примерно в 1,9 раза. Стоимость одного шага декодирования складывается из двух частей: L -зависимая (чтение KV-кэша из видеопамяти, растёт линейно с L) и L -независимая (проекция $Q/K/V/O$ и полносвязные слои MLP, работающие с фиксированными матрицами весов). При росте контекста с 4K до 32K объём чтений KV-кэша возрастает в восемь раз, но за счёт амортизации L -независимой компонентой суммарная стоимость шага растёт лишь в $\sim 1,9$ раза. При дальнейшем росте L доля KV-кэшевой части начнёт доминировать, и деградация будет ближе к линейной по L , что и мотивирует поиск архитектур с компактным кэшем. Квадратичность Softmax-внимания относится к этапу предварительного заполнения, а не к декодированию. Пиковое потребление видеопамяти держится около 22,6 ГБ с небольшим запасом до лимита GPU.

Ожидаемый выигрыш в идеальной гибридной реализации. В идеальной реализации Selective SW ускоряется не вся модель, а прежде всего та часть вычислений, которая связана с чтением KV-кэша и операцией внимания. Для этой части можно использовать ту же оценку, что и для объёма KV-кэша:

$$R_{\text{attn}} = p_{\text{kv}} + (1 - p_{\text{kv}}) \frac{K}{L}, \quad (17)$$

где p_{kv} — доля защищённых KV-групп, а $K = W + S = 516$. Величина R_{attn} показывает, какая доля attention/KV-работы остаётся по сравнению с полным вниманием. Например, если $R_{\text{attn}} = 0,5$, это означает, что гибридная реализация выполняет только половину attention/KV-работы исходной модели.

Поскольку скорость обратно пропорциональна объёму выполняемой работы, потенциальное ускорение attention/KV-части равно обратной величине:

$$\text{Speedup}_{\text{attn}} = \frac{1}{R_{\text{attn}}}. \quad (18)$$

Иными словами, если исходная модель выполняет условно одну единицу attention/KV-работы, то гибридная реализация выполняет R_{attn} единиц работы. Поэтому отношение времени работы исходной реализации ко времени работы гибридной реализации равно $1/R_{\text{attn}}$.

Если использовать верхнюю оценку доли защищённых KV-групп, то для маски default имеем:

$$p_{kv}^{\max} = \frac{173}{288} \approx 0,601. \quad (19)$$

Для масок medium и safe строгая верхняя граница равна 100%, поскольку при неудачном распределении защищённые Q -головы могут затронуть все KV-группы.

Таблица 5: Нижняя оценка теоретического ускорения attention/KV-части в идеальной реализации Selective SW при верхней оценке доли защищённых KV-групп. Ускорение всей модели будет меньше, так как MLP и проекции не ускоряются.

Маска	$p_{kv}^{\max}$	$L = 4K$	$L = 8K$	$L = 32K$
default	$\sim 60,1\%$	$1,54\times$	$1,60\times$	$1,65\times$
medium	100%	$1,00\times$	$1,00\times$	$1,00\times$
safe	100%	$1,00\times$	$1,00\times$	$1,00\times$

Для маски default при длине контекста $L = 32768$ получаем:

$$R_{\text{attn}} = \frac{173}{288} + \left(1 - \frac{173}{288}\right) \frac{516}{32768} \approx 0,607. \quad (20)$$

Следовательно, нижняя оценка ускорения attention/KV-части равна:

$$\text{Speedup}_{\text{attn}} = \frac{1}{0,607} \approx 1,65. \quad (21)$$

Таким образом, по строгой верхней оценке объёма KV-кэша заметный гарантируемый прирост attention/KV-части получается только для наиболее агрессивной маски default. На длине 32K идеальная attention/KV-часть для этой маски должна ускоряться не менее чем примерно в 1,65 раза. Для масок medium и safe такая верхняя оценка не гарантирует ускорения, хотя фактическое ускорение может появиться, если защищённые Q-головы в реальной маске сосредоточены в меньшем числе KV-групп.

Важно, что это оценка только для attention/KV-части. Ускорение всей модели будет меньше, поскольку значительная часть времени тратится не на attention, а на MLP, линейные проекции, нормировки и служебные операции инференса.

На NIAH полная модель даёт 100% во всех 12 ячейках — это задаёт верхнюю границу качества извлечения для всех последующих гибридных конфигураций.

5.2. Валидация масок на NIAH

Чтобы проверить каждую маску, отмеченные ею головы оставлены в Softmax-режиме, остальные переведены в скользящее окно $W = 512$. Для абляционного исследования дополнительно строится случайная маска того же размера. Это даёт ответ на вопрос, действительно ли отобранные головы несут долгосрочную информацию, или результат определяется только количеством защищённых голов.

Результаты валидации приведены в табл. 6.

Из табл. 6 следуют три вывода:

1. Конфигурация default (15%) теряет иглу из начала длинного контекста при $L_{in} \geq 16K$ — бюджета голов недостаточно.
2. Конфигурация medium (32%) даёт 100% во всех 12 ячейках, тогда как случайная маска того же размера на $depth \in \{0; 0,5\}$ не справляется. Следовательно, отобранные головы действительно несут долгосрочную информацию, а результат не объясняется только количеством защищённых голов.
3. Конфигурация safe (50%) также даёт 100% везде, но и случайная маска такого размера даёт 100% — бюджет избыточен, целенаправленный

Таблица 6: NIAH-точность при защите голов извлечения: целенаправленная маска / случайная маска того же размера. Полужирным выделены проседания от 100%.

L_{in}	depth	default (15%)	medium (32%)	safe (50%)
4K	0,00	100% / 0%	100% / 0%	100% / 100%
4K	0,50	100% / 0%	100% / 0%	100% / 100%
4K	1,00	100% / 100%	100% / 100%	100% / 100%
8K	0,00	100% / 0%	100% / 0%	100% / 100%
8K	0,50	100% / 0%	100% / 0%	100% / 100%
8K	1,00	100% / 100%	100% / 100%	100% / 100%
16K	0,00	0% / 0%	100% / 0%	100% / 100%
16K	0,50	100% / 0%	100% / 0%	100% / 100%
16K	1,00	100% / 100%	100% / 100%	100% / 100%
32K	0,00	0% / 0%	100% / 0%	100% / 100%
32K	0,50	100% / 0%	100% / 0%	100% / 100%
32K	1,00	100% / 100%	100% / 100%	100% / 100%

отбор уже не информативен.

Для дальнейшего сравнения архитектур выбрана конфигурация default (15% защищённых голов) — наиболее агрессивный бюджет, позволяющий ярче выявить различия между конфигурациями. Для практического применения рекомендуется конфигурация medium (32%) — минимальный бюджет, удовлетворяющий критерию $\geq 95\%$ точности и при этом заметно превосходящий случайный отбор.

5.3. Сравнение четырёх архитектур

Результаты сравнения четырёх конфигураций по NIAH-точности приведены в табл. 7, по перплексии на WikiText-2 — в табл. 8, сводное сравнение — в табл. 9.

Визуальное представление табл. 7 в виде тепловых карт (рис. 2) наглядно показывает, где каждая конфигурация утрачивает способность к извлечению: Original Qwen3 полностью зелёная, Selective SW теряет лишь две ячейки в левом верхнем углу, Partial LoLCATs демонстрирует неоднородные провалы, а Full LoLCATs почти целиком красная за исключением $\text{depth} = 1$.

На рис. 3 хорошо видно, что при $\text{seq_len} = 512$ (синие столбцы) все четыре конфигурации практически неразличимы: скользящее окно полностью покрывает контекст. При $\text{seq_len} = 2048$ (красные столбцы) Selective SW остаётся близким к эталону, тогда как Partial и Full LoLCATs демонстрируют

Таблица 7: NIAH-точность четырёх конфигураций. Каждая ячейка — 3 прогона. Полужирным выделены провалы относительно эталона.

L_{in}	depth 0,00	depth 0,50	depth 1,00	среднее
Original Qwen3				
4K–32K	100%	100%	100%	100%
Selective SW (15% голов извлечения, $W = 512$)				
4K	100%	100%	100%	100%
8K	100%	100%	100%	100%
16K	0%	100%	100%	67%
32K	0%	100%	100%	67%
Partial LoLCATs (15% голов извлечения, после Stage-1)				
4K	100%	0%	100%	67%
8K	0%	0%	100%	33%
16K	100%	0%	100%	67%
32K	0%	0%	0%	0%
Full LoLCATs (0% голов извлечения, после Stage-1)				
4K	0%	0%	100%	33%
8K	0%	0%	0%	0%
16K	0%	0%	100%	33%
32K	0%	0%	100%	33%

Таблица 8: Перплексия на WikiText-2 (датасет не использовался при адаптации). Ниже — лучше.

Конфигурация	seq=512	seq=2048	Δ vs orig. @2048
Original Qwen3	16,17	13,54	—
Selective SW	16,18	14,04	+3,7%
Partial LoLCATs	16,16	28,55	+110,9%
Full LoLCATs	16,14	33,07	+144,2%

резкий рост перплексии.

Таблица 9: Сводное сравнение конфигураций.

Критерий	Original	Selective SW	Partial LoL.	Full LoL.
NIAH (средняя)	100%	~83%	~42%	~25%
PPL @ seq=2048	13,54	14,04	28,55	33,07
Доп. параметры	0	0	~415M	~415M
Адаптация	нет	нет	да (Stage-1)	да (Stage-1)

Почему прототип не использовался для сравнения скорости. Теоретические оценки объёма KV-кэша и возможного прироста пропускной способности относятся к идеальной реализации. Под идеальной реализацией здесь понимается такая схема инференса, в которой защищённые KV-группы и потоковые KV-группы обрабатываются отдельно. Защищённые KV-группы сохраняют

NIAH-точность четырёх конфигураций

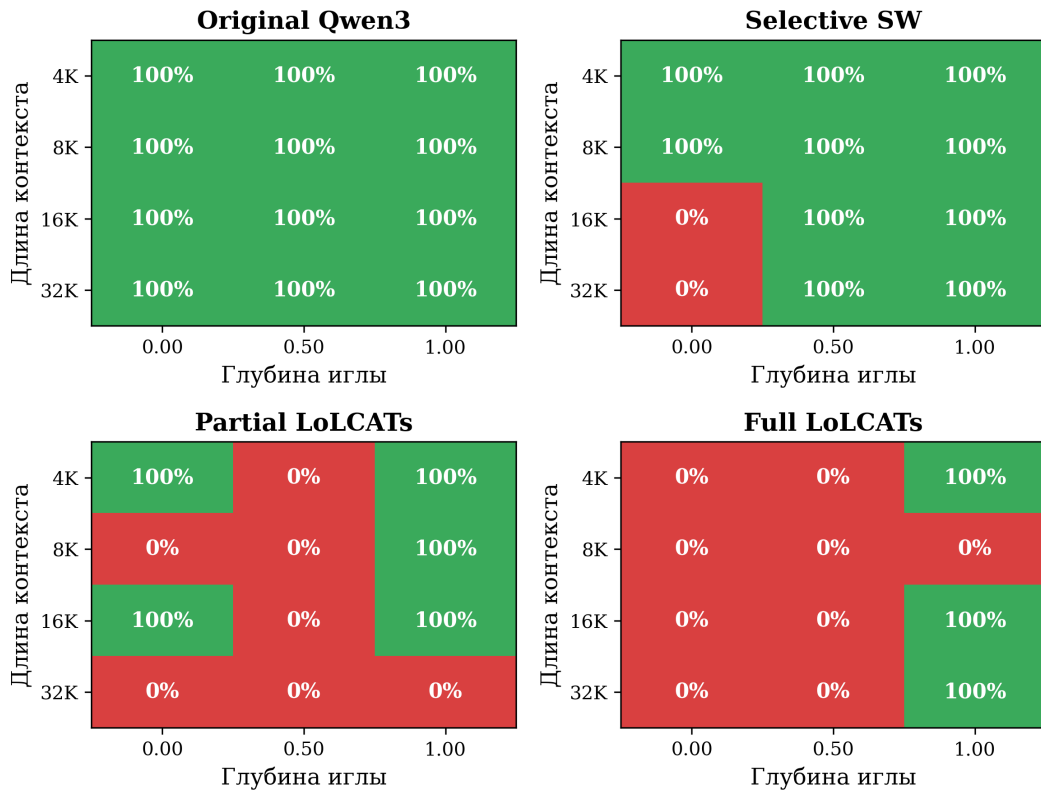


Рис. 2: Тепловые карты NIAH-точности четырёх конфигураций. Зелёный — 100%, красный — 0%. Оси: длина контекста (вертикальная) и глубина иглы (горизонтальная).

полный Softmax-режим и полный KV-кэш, а потоковые KV-группы переводятся в субквадратичный режим и хранят только локальное окно и токены-аттракторы. В такой реализации каждая KV-группа должна вычисляться ровно один раз.

Прототипы, использованные в работе, решают другую задачу: они проверяют качество выбранных масок и устойчивость частичной гибридизации, но не являются финальной производительной реализацией для инференса. В реализации Selective Sliding Window сначала вычисляется скользящее окно для всех Q -голов, включая головы извлечения, а затем головы извлечения повторно вычисляются в полном Softmax-режиме. После этого результат полного внимания записывается поверх результата скользящего окна. Поэтому фактическая схема вычислений отличается от идеальной: вместо отдельной обработки защищённых и потоковых групп прототип выполняет скользящее окно для всех голов и дополнительно пересчитывает защищённую часть.

Это особенно важно на коротких и средних длинах контекста. Если

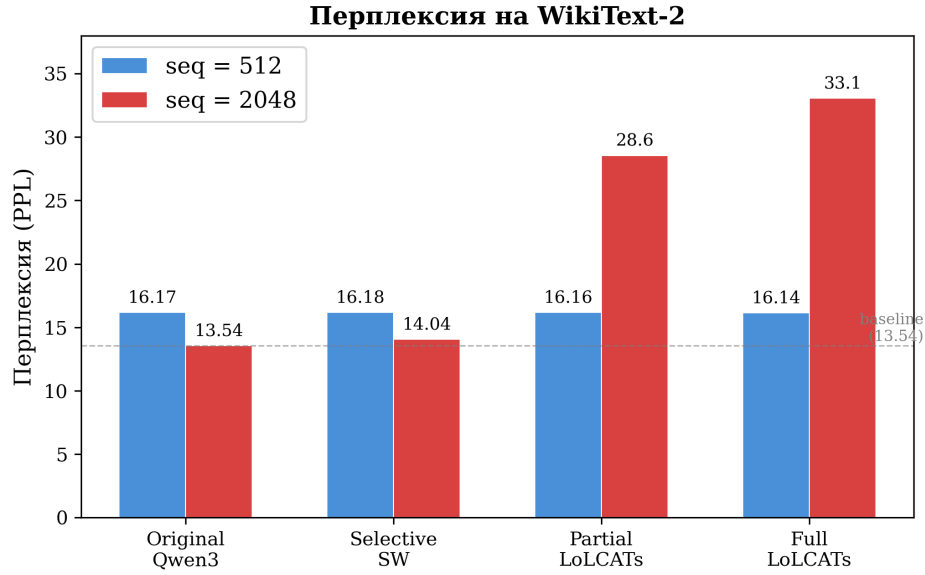


Рис. 3: Перплексия на WikiText-2 для четырёх конфигураций при $\text{seq_len} \in \{512, 2048\}$. Пунктирная линия — эталон (13,54).

длина контекста сравнима с размером окна $W = 512$, то скользящее окно почти не уменьшает объём вычислений по сравнению с полным вниманием, но добавляет дополнительный проход и операции копирования. В результате прототип может работать медленнее исходной модели с оптимизированным FlashAttention, несмотря на то что идеальная схема должна уменьшать объём работы с KV-кэшем.

Для Partial LoLCATs накладные расходы ещё выше. В этой конфигурации потоковые головы получают не только скользящее окно, но и дополнительную ветвь управляемого линейного внимания GLA. Поэтому к вычислению скользящего окна и повторному вычислению голов извлечения добавляются признаковые отображения ϕ_q, ϕ_k , вентили забывания и рекуррентное состояние линейного внимания. Такая схема полезна для проверки качества гибридизации, но без специализированных вычислительных ядер она не отражает нижнюю теоретическую стоимость метода.

Отдельным ограничением является организация KV-кэша. Для достижения теоретической экономии памяти необходимо физически хранить полный KV-кэш для защищённых KV-групп и укороченный KV-кэш для потоковых KV-групп отдельно. В прототипе смешанные случаи обрабатываются менее эффективно: часть преимуществ теряется из-за копирования тензоров, дополнения нулями до полной длины, дополнительных вызовов вычислительных ядер и преобразований форматов данных. Поэтому экспериментальные

результаты по качеству следует интерпретировать как проверку гипотезы о выборе голов извлечения, а не как итоговый замер пропускной способности гибридных конфигураций.

Таким образом, отсутствие ускорения у прототипа не противоречит теоретической оценке. Теоретический выигрыш появляется только в оптимизированной реализации, где защищённые и потоковые KV-группы вычисляются и хранятся раздельно. По строгой верхней оценке для маски `default` на длине 32K attention/KV-часть должна ускоряться не менее чем в 1,65 раза. Однако ускорение всей модели будет меньше, поскольку полносвязные слои, линейные проекции, нормировки и служебные операции инференса не переводятся в субквадратичный режим.

Рис. 4 визуализирует строку «NIAH (средняя)» из табл. 9 и наглядно демонстрирует масштаб разрыва между конфигурациями: от 100% у эталона до 25% у Full LoLCATs.

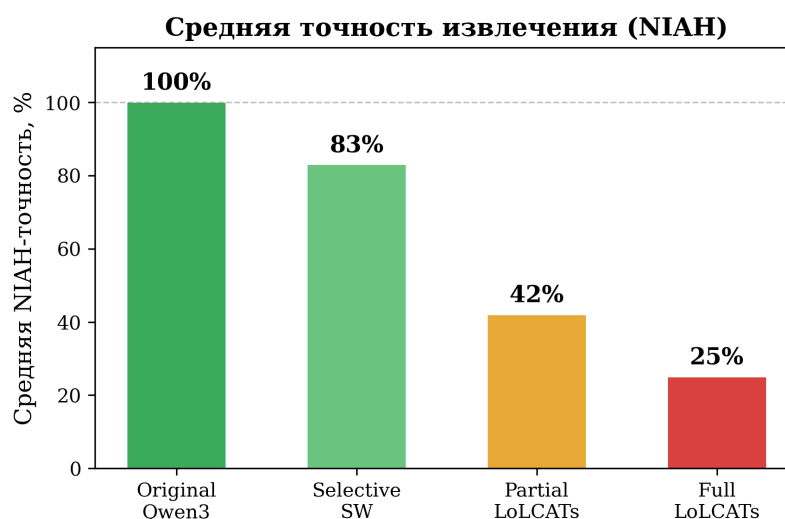


Рис. 4: Средняя NIAH-точность четырёх конфигураций по всем 12 ячейкам.

В табл. 7 и табл. 9 указан Selective SW с 15%-маской (`default`), как в основном эксперименте. Конфигурация Selective SW с 32%-маской (`medium`) даёт 100% NIAH-точности во всех 12 ячейках — см. валидацию маски в табл. 6 — и рекомендуется как практический выбор (см. §6.2).

Original Qwen3. Верхняя граница качества: 100% NIAH во всех 12 ячейках и перплексия 13,54 на WikiText-2 (seq=2048). Все последующие конфигурации измеряются как деградация относительно этого эталона.

Selective SW. 100% точности на 10 из 12 ячеек NIAH (табл. 7). Проседание наблюдается только в точке ($L_{in} \geq 16K$, $depth = 0$); это граница применимости 15%-маски, для которой необходима конфигурация medium (табл. 6). Перплексия (табл. 8) деградирует на 3,7%: головы извлечения обеспечивают доступ к дальнему контексту, головы в скользящем окне — к локальному.

Partial LoLCATs. В среднем уступает Selective SW: средняя NIAH-точность ~42% против ~83% (табл. 9), перплексия примерно в 2 раза хуже (табл. 8). По ячейкам картина неоднородна: в 6 ячейках из 12 Selective SW строго лучше, в 5 результаты эквивалентны, в одной (16K, $depth = 0$) Partial LoLCATs лучше (100% против 0%). Показателен случай (16K, 0,5): чистое скользящее окно даёт 100%, а добавление линейной GLA-ветви — 0%. Линейная ветвь не добавляет новой информации и в большинстве режимов нарушает координацию между головами извлечения и потоковыми головами, размывая выход.

Full LoLCATs. 0% во всех 8 ячейках с $depth < 1$ (табл. 7, рис. 2) — модель не может обратиться к игле за пределами локального скользящего окна. Перплексия (табл. 8, рис. 3) деградирует в 2,44 раза, то есть страдает даже базовое предсказание следующего токена, а не только редкое извлечение из дальнего контекста.

5.4. Диагностика обученных параметров LoLCATs

После 2000 шагов передачи внимания проверено, как изменились обучаемые параметры. Результаты приведены в табл. 10.

По существу изменились только признаковые отображения ϕ_q , ϕ_k — функции, аппроксимирующие Softmax-ядро в линейной ветви. Три остальных группы параметров остались около начальной инициализации:

- **Параметр смешивания** `mix_logit` (управляет балансом α между скользящим окном и линейной GLA-ветвью) отклонился от начального значения не более чем на $\pm 0,03$. Это означает, что выход слоя по-прежнему определяется почти исключительно скользящим окном ($\alpha \approx 0,993$).
- **Веса проекции вентиляей** `g_proj.weight` (проекция, формирующая для GLA сигналы о том, какую долю накопленного рекуррентного состоя-

ния сохранять на каждом шаге) остались близки к нулю — проекция фактически не обучилась.

- **Смещения проекции вентилей** `g_proj.bias` не изменились (≈ 0), то есть вентили зафиксированы на нейтральном значении $\sigma(0) = 0,5$.

Причина заключается в конфигурации: при длине последовательности $1024 = 2W$ скользящее окно покрывает половину последовательности по позициям и почти весь контент, поэтому ветвь скользящего окна хорошо приближает выход самостоятельно, а обучающий сигнал на параметр смешивания и вентили забывания практически отсутствует.

Таблица 10: Состояние обучаемых параметров LoLCATs после 2000 шагов.

Параметр	Поведение	Интерпретация
<code>mix_logit</code>	$\pm 0,03$ от <code>init</code>	баланс SW/GLA не сдвинулся
<code>g_proj.weight</code>	$\ W\ \approx 0$	проекция вентилей не обучилась
<code>g_proj.bias</code>	≈ 0 (не изм.)	вентили на $\sigma(0) = 0,5$
<code>phi_q.W</code> , <code>phi_k.W</code>	$\ \phi - I\ \in [0,1; 2,5]$	признак. отображения обучились

5.5. Сравнение с существующими результатами

Полученные результаты согласуются с известными ограничениями линейного внимания. В работе [3] линейризованная Llama 3 8B даёт 0% на задаче извлечения ключа из контекста — то же качественное поведение наблюдается у Full LoLCATs в данной работе (0% на NIAH во всех ячейках с $\text{depth} < 1$).

LAWCAT [7] достигает более 90% на задаче извлечения для Mistral 7B, но ценой полноценной адаптации новых параметров и дополнительных архитектурных компонентов (причинная свёртка Conv1D, нормированное управляемое внимание). В условиях настоящей работы (около 150К токенов обучающих данных вместо более 40 млн у LoLCATs и LAWCAT) достичь такого результата невозможно, и не требующий адаптации Selective SW оказывается практически оптимальным. Вместе с тем следует отметить, что сравнение не является полностью корректным: при сопоставимом объёме обучающих данных конфигурации с адаптацией (Partial и Full LoLCATs) могли бы показать существенно лучшие результаты.

Сравнение с NBL [6] носит косвенный характер: в модели DeepSeek-R1-Distill-Llama-8B NBL заменяет до 12 слоёв с потерей точности менее 1% на задачах рассуждения, но на точечном извлечении не тестируется.

Метрики разнотипны (NBL оценивается по точности на задачах рассуждения, настоящая работа — по перплексии и NIAH), поэтому прямое сопоставление чисел некорректно. Тем не менее можно утверждать: Selective SW сохраняет точечное извлечение (83% NIAH при 15%-маске, 100% при 32%-маске) при деградации перплексии лишь +3,7% относительно полной модели. У NBL аналогичной оценки на NIAH нет; косвенно это свидетельствует в пользу функционального критерия отбора (концентрация голов извлечения) перед статистическим (ССА), если задача требует извлечения из дальнего контекста.

6. Заключение

6.1. Подтверждение основной гипотезы

Главный вопрос работы был сформулирован так: лучше ли частичная гибридизация, чем полная. Результаты экспериментов отвечают на него однозначно — да, частичная заметно лучше полной по обеим группам метрик качества:

- **NIAH:** любая частичная конфигурация (Selective SW или Partial LoLCATs) опережает полную (Full LoLCATs) по средней точности (табл. 9, рис. 4). Лучшая из них — Selective SW — даёт около 83% против ~25% у Full LoLCATs, разрыв составляет более 3×. На задачах с глубоко расположенной иглой ($\text{depth} < 1$) Full LoLCATs показывает 0% во всех ячейках (табл. 7, рис. 2): полная гибридизация структурно неспособна к точечному извлечению.
- **Перплексия (WikiText-2, seq=2048):** частичная гибридизация (Selective SW) деградирует на 3,7% относительно полной модели, полная (Full LoLCATs) — на 144% (табл. 8, рис. 3). Относительная деградация различается более чем в 30 раз.

Дополнительно установлено, что при увеличении бюджета защищённых голов до 32% (маска medium) частичная гибридизация доходит до 100% NIAH-точности во всём диапазоне $L_{in} \in [4K, 32K]$ (табл. 6), то есть имеет естественный способ масштабирования качества, отсутствующий у полной гибридизации.

Тот факт, что обе частичные конфигурации опережают полную, важен сам по себе: преимущество не является побочным эффектом адаптации или её отсутствия. Определяющий фактор — именно стратегия гибридизации по функциональному критерию: сохранение голов извлечения в полном Softmax-режиме при переводе остальных в субквадратичный режим. В среднем Partial LoLCATs (с адаптацией новых параметров) уступает Selective SW (без адаптации) по перплексии примерно вдвое (28,55 против 14,04 при эталоне 13,54) и по NIAH (~42% против ~83%). Добавление обучаемой линейной GLA-ветви для 85% потоковых голов не улучшает, а ухудшает качество по сравнению с чистым скользящим окном на тех же 85% голов.

С точки зрения эффективности частичная гибридизация внимания даёт выигрыш за счёт уменьшения объёма KV-кэша. В идеальной реализации Selective Sliding Window головы извлечения сохраняют полный Softmax-режим, а потоковые головы используют скользящее окно. Поэтому для потоковых KV-групп хранится не весь контекст, а только $W + S = 516$ позиций.

Для маски default, где защищено около 15% Q -голов, строгая верхняя оценка доли защищённых KV-групп равна $173/288 \approx 60,1\%$. При длине контекста 32К это даёт верхнюю оценку объёма KV-кэша около 60,7% от исходной модели, то есть примерно 2,73 GiB вместо 4,50 GiB для Qwen3-4B при размере пакета 1 и формате BFloat16. Соответственно, нижняя оценка ускорения attention/KV-части составляет $1/0,607 \approx 1,65$ раза.

Для масок medium и safe строгая верхняя граница равна полному KV-кэшу, поскольку в худшем случае защищённые Q -голова могут затронуть все KV-группы. Это не исключает фактической экономии, но она должна рассчитываться по реальному распределению маски.

Таким образом, более агрессивная маска default даёт больший потенциальный выигрыш по памяти и скорости, но хуже сохраняет качество точечного извлечения. Более безопасные маски лучше сохраняют качество, но дают меньшую гарантированную экономию.

6.2. Практическая рекомендация

Из четырёх протестированных конфигураций для практического применения рекомендуется **Selective Sliding Window** — частичная гибридизация, в которой головы извлечения оставлены в полном Softmax-внимании, а остальные переведены на скользящее окно. Основания:

- качество ближе всего к эталону (+3,7% перплексии, 100% NIAH в 10/12 ячейках при 15% голов извлечения; полное покрытие 12/12 достигается переходом на маску medium с 32% голов извлечения, валидация в табл. 6);
- ноль новых обучаемых параметров;
- отсутствие этапа адаптации — маска голов извлечения строится один раз на синтетической последовательности, и модель сразу готова к использованию;

- в отличие от Partial LoLCATs, не требует реализации и отладки GLA-ядер и процедуры передачи внимания, которая при ограниченном бюджете данных не достигает сходимости (см. табл. 10);
- в отличие от Full LoLCATs, не утрачивает способности к точечному извлечению.

Конкретный рецепт: для Qwen3-4B и аналогичных моделей выбирается маска `medium` (топ-30% по индукционной оценке + топ-2% по эхо-оценке, около 32% защищённых голов); все головы извлечения остаются в полном Softmax-внимании, остальные переводятся в скользящее окно $W = 512$ с 4 токенами-аттракторами. Такой режим сохраняет всю NIAH-точность эталона и практически всю перплексию.

Список литературы

- [1] Vaswani, A. Attention Is All You Need / A. Vaswani, N. Shazeer, N. Parmar [и др.] // Advances in Neural Information Processing Systems 30 (NeurIPS). — 2017. — P. 5998–6008.
- [2] Katharopoulos, A. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention / A. Katharopoulos, A. Vyas, N. Pappas, F. Fleuret // Proceedings of the 37th International Conference on Machine Learning (ICML). — 2020. — P. 5156–5165.
- [3] Zhang, M. LOLCATS: On Low-Rank Linearizing of Large Language Models / M. Zhang, S. Arora, R. Chalamala [и др.] // International Conference on Learning Representations (ICLR). — 2025.
- [4] Hu, E. J. LoRA: Low-Rank Adaptation of Large Language Models / E. J. Hu, Y. Shen, P. Wallis [и др.] // International Conference on Learning Representations (ICLR). — 2022.
- [5] Zhang, M. The Hedgehog & the Porcupine: Expressive Linear Attentions with Softmax Mimicry / M. Zhang, K. Bhatia, H. Kumbong, C. Ré // International Conference on Learning Representations (ICLR). — 2024.
- [6] Erdogan, M. Efficient Large Language Model Inference with Neural Block Linearization / M. Erdogan, F. Tonin, V. Cevher // Advances in Neural Information Processing Systems 38 (NeurIPS). — 2025.
- [7] Liu, Z. LAWCAT: Efficient Distillation from Quadratic to Linear Attention with Convolution across Tokens for Long Context Modeling [Электронный ресурс] / Z. Liu, Z. Wang, Z. Huang [и др.]. — 2025. — URL: <https://arxiv.org/abs/2509.18467> (дата обращения: 20.05.2026).
- [8] Tang, H. RazorAttention: Efficient KV Cache Compression Through Retrieval Heads [Электронный ресурс] / H. Tang, Y. Lin, J. Lin [и др.]. — 2024. — URL: <https://arxiv.org/abs/2407.15891> (дата обращения: 20.05.2026).
- [9] Xiao, G. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads [Электронный ресурс] / G. Xiao, J. Tang, J. Zuo [и др.]. — 2025. — URL: <https://arxiv.org/abs/2509.18467> (дата обращения: 20.05.2026).

др.]. — 2024. — URL: <https://arxiv.org/abs/2410.10819> (дата обращения: 20.05.2026).

- [10] Yang, S. Gated Linear Attention Transformers with Hardware-Efficient Training [Электронный ресурс] / S. Yang, B. Wang, Y. Shen [и др.]. — 2023. — URL: <https://arxiv.org/abs/2312.06635> (дата обращения: 20.05.2026).
- [11] Press, O. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation / O. Press, N. A. Smith, M. Lewis // International Conference on Learning Representations (ICLR). — 2022.
- [12] Su, J. RoFormer: Enhanced Transformer with Rotary Position Embedding [Электронный ресурс] / J. Su, Y. Lu, S. Pan [и др.]. — 2021. — URL: <https://arxiv.org/abs/2104.09864> (дата обращения: 20.05.2026).
- [13] Qwen Team. Qwen3 Technical Report [Электронный ресурс]. — 2025. — URL: <https://arxiv.org/abs/2505.09388> (дата обращения: 20.05.2026).
- [14] Olsson, C. In-context Learning and Induction Heads [Электронный ресурс] / C. Olsson, N. Elhage, N. Nanda [и др.]. — 2022. — URL: <https://arxiv.org/abs/2209.11895> (дата обращения: 20.05.2026).
- [15] Wu, W. Retrieval Head Mechanistically Explains Long-Context Factuality [Электронный ресурс] / W. Wu, Y. Wang, G. Xiao [и др.]. — 2024. — URL: <https://arxiv.org/abs/2404.15574> (дата обращения: 20.05.2026).
- [16] Nguyen, C. V. Lizard: An Efficient Linearization Framework for Large Language Models [Электронный ресурс] / C. V. Nguyen, H. Nguyen, R. Zhang [и др.]. — 2026. — URL: <https://arxiv.org/abs/2507.09025> (дата обращения: 20.05.2026).